BPSC TRE 4.0

Computer Science (Class 11–12)

Day 5

1. Computer Fundamentals > Operating Systems

2026-11-01

BPSC TRE 4.0 Computer Science — Day 5 — 1. Computer Fundamentals > Operating Systems — 2026-11-01

Notes • 35 Questions • Answer Key + व्याख्या

OS Part 3/4 — Sync, Deadlock

1 पढ़ने से पहले

- आज Part 3/4 है। पिछले दिनों में OS (Operating System) का परिचय, CPU (Central Processing Unit) scheduling, memory और disk पढ़ा। आज: Process Synchronization, Deadlock, Process/Thread और UNIX की कुछ बातें।
- पहले एक बात: Process = चलता हुआ program। Thread = process के अंदर का छोटा काम-धागा।
- आज के कुछ सवाल (disk, paging) पिछले notes के topics के हैं; उनके लिए Part 2 के notes देखो।
- इस topic से TRE (Teacher Recruitment Exam) 1-3 में आज नया सीधा सवाल नहीं मिला; सब पहले के दिनों में आ चुके हैं।

2 1. आसान भाषा में (Synchronization)

- जब दो या ज़्यादा processes एक ही चीज़ (जैसे एक variable, एक file) एक साथ बदलें तो गड़बड़ हो सकती है। इसे Race Condition कहते हैं (नतीजा इस पर निर्भर कि कौन पहले पहुँचा)।
- Critical Section (CS) = code का वह हिस्सा जहाँ shared चीज़ छुई जाती है। नियम: एक समय पर एक ही process CS में रहे।
- इस नियम को निभाने के तरीकों को Synchronization कहते हैं: Semaphore, Monitor, Message passing।

3 2. रोज़मर्रा का example

- एक ही ATM (Automated Teller Machine) केबिन में एक बार में एक ही आदमी जाता है = Mutual Exclusion।
- बाहर वाला अंदर वाले को नहीं रोकता; केबिन खाली हो तो अगला तुरंत जाए = Progress।
- कोई आदमी हमेशा बाहर खड़ा न रहे, उसकी बारी आए = Bounded Waiting।

- अगर नियम हो 'पहले तुम, फिर मैं और तुम आओ ही नहीं तो मैं फँस जाऊँगा = Progress टूटा।

④ 3. Critical Section की 3 शर्तें और tools

- Mutual Exclusion: एक समय एक process।
- Progress: CS खाली है तो जो जाना चाहता है उसे जाने दो; जो CS में जाना नहीं चाहता वह दूसरे को न रोके।
- Bounded Waiting: इंतज़ार की सीमा हो, कोई हमेशा न रुके।
- कोई assumption नहीं कि CPU कितने हैं या कितनी तेज़ हैं।
- Semaphore: एक integer। P / wait: मान । घटाओ (0 से कम हो तो रुको)। V / signal: मान । बढ़ाओ।
  - → Binary semaphore: 0 या । (lock जैसा)। Counting semaphore: कई units वाली चीज़ों के लिए।
- Monitor: language-level package; procedures + variables + data एक साथ, एक समय में एक ही process अंदर। Java में synchronized methods यही काम करते हैं।
- Message passing: send और receive से बातचीत।

```
wait(S): S = S - 1   // S<0 □□ □□ process □□□□□□□□ □□□
signal(S): S = S + 1
// Java: public synchronized void deposit(int amt) { ... }
```

⑤ Worked example (Semaphore)

- सवाल: semaphore का मान 7 था; 20 P और x V हुए; अंत में 5। x?
- Step 1: P से मान घटता है: 7 − 20 = −13।
- Step 2: V से बढ़ता है: −13 + x = 5।
- Step 3: x = 18।
- Trap: मान negative हो सकता है; negative संख्या = इंतज़ार करते processes।

⑥ 4. Deadlock — आसान भाषा में

- Deadlock = कुछ processes हमेशा के लिए अटक जाएँ, क्योंकि हर कोई

वह चीज़ माँग रहा है जो दूसरे के पास है।

- Example: चौराहे पर चारों तरफ से गाड़ियाँ बीच में फँस गईं; कोई पीछे नहीं हट सकता।
- 4 शर्तें (चारों साथ हों तभी deadlock): Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait।
- Prevention: इनमें से एक शर्त तोड़ दो। Havender का तरीका deadlock prevention का है।
- Avoidance: Banker's algorithm; system को safe state में रखो।
- Recovery: process रोको (terminate) या resource छीनो।
- Cascading termination: parent खत्म तो OS child को भी खत्म कर दे।

(7) Worked example (formula)

- Formula: n processes, हर को अधिकतम m resource, कुल R। Deadlock-free अगर $R \geq n \times (m-1) + 1$।
- सवाल: 12 resources, हर process को 4 → $12 \geq n \times 3 + 1 \rightarrow n \leq 3$।
- सवाल: 6 tape drives, हर को 2 → $6 \geq n \times 1 + 1 \rightarrow n \leq 5$।
- क्यों? सबसे बुरा हाल: हर को (m−1) मिल गए और सब 1 और का इंतज़ार कर रहे हैं। उसके ऊपर 1 और बचे तो कोई एक पूरा हो जाएगा।

(8) 4. Exam Point + Trap

- 2PL (Two-Phase Locking) deadlock-free नहीं है; Timestamp ordering deadlock-free और conflict-serializable है (UGC NET (University Grants Commission National Eligibility Test) में दो बार पूछा)।
- Critical Section की शर्त में 'बाहर वाला दूसरों को रोक सकता है' शर्त नहीं है।
- Conservative 2PL deadlock-free होता है (पर सब locks पहले लेने पड़ते हैं)।
- Trap: formula में (m−1) भूल जाना।

⑨ 5. BPSC ऐसे पूछता है (TRE 1.0/2.0/3.0)

- आज के इस part का सीधा TRE सवाल नहीं मिला।
- TRE की शैली: परिभाषा वाला सवाल और 'कौन सा नहीं है' वाला। इसलिए परिभाषाएँ याद रखो और 'नहीं है' वाले options ध्यान से पढ़ो।

⑩ 6. NET level में ऐसे आता है

- Type 1: Statement I/II (2PL vs Timestamp): देखो किसमें wait है; wait है तो deadlock संभव।
- Type 2: Numerical: semaphore का मान, deadlock-free maximum n।
- Type 3: Match the list: Stack algorithm = page replacement; Elevator = disk; Priority = CPU scheduling; Havender = deadlock।
- Type 4: Code देखकर Critical Section property पहचानना (बारी-बारी का नियम = Progress नहीं)।

⑪ 7. याद रखो

- CS की 3 शर्तें: Mutual Exclusion, Progress, Bounded Waiting।
- Deadlock की 4 शर्तें; formula $R \geq n(m-1)+1$।
- Semaphore: P घटाता, V बढ़ाता। Monitor = high-level।

Process, Thread, Kernel

① 1. आसान भाषा में

- Process = चलता program। नया process बनाने का तरीका: UNIX में fork() system call।
- fork() के बाद parent और child के पास stack और heap की अलग-अलग कॉपी होती है, पर shared memory segment दोनों का साझा रहता है।
- Linux में clone() से बनता है; इसमें flags देकर तय करते हैं कितना share हो। यही thread बनाने का आधार है।
- Pthreads = POSIX (Portable Operating System Interface) thread

का specification (नियमों की किताब), implementation नहीं। इसे Solaris, Linux, Mac OS X, Tru64 UNIX लागू करते हैं।

- Microkernel = बहुत छोटा kernel, जो client program और user-space services (file server आदि) के बीच communication कराता है।

② 2. रोज़मर्रा का example

- Parent दुकान का मालिक; fork() से मालिक ने अपनी पूरी दुकान की कॉपी बना ली (child), पर दोनों एक ही तिजोरी (shared memory) इस्तेमाल करते हैं।
- Microkernel = ऑफिस का receptionist जो सिर्फ बात आगे पहुँचाता है, काम विभाग (services) करते हैं।

③ 3. step by step

- fork(): child को parent जैसा process मिलता है; return value parent को child का PID (Process IDentifier), child को 0।
- clone(flags): flags से share करने वाली चीज़ें चुनो → thread जैसा।
- Cascading termination: parent खत्म, तो child भी खत्म (कुछ OS में)।
- Zombie: खत्म हो चुका process जिसकी status parent ने अभी नहीं पढ़ी।
- Orphan: parent खत्म हो गया, child चल रहा है (UNIX में init उसे अपना लेता है)।

④ 4. Exam Point + Trap

- fork में shared = shared memory segment; heap और stack नहीं।
- Pthreads = specification, implementation नहीं; standard IEEE 1003.1c।
- Trap: fork() और clone() को एक न समझें; clone में flags होते हैं।
- Windows 2000 का non-portable हिस्सा: Virtual Memory Management (UGC NET key)।
- Disk partition manager (shrink/extend) पहली बार Windows Vista में।

⑤ 5. BPSC ऐसे पूछता है (TRE 1.0/2.0/3.0)

- TRE में Thread का सवाल आ चुका है: 'thread किसके साथ share करता है'। जो चीज़ thread की अपनी है (stack, registers, PC) वह share नहीं होती; code, data, files share होते हैं।
- ऐसे सीधे definition सवाल पहचान कर 'अपना बनाम साझा' की list याद रखो।

⑥ 6. NET level में ऐसे आता है

- Type: 'कौन सा statement सही नहीं' जैसे KILL signal blockable (गलत), Pthreads implementation (गलत)।
- Type: system call पहचानना: flags हों तो clone()।
- Type: Definition match: Cascading termination, Microkernel।

⑦ 7. याद रखो

- fork = copy; clone = flags से share।
- Pthreads = specification।
- Microkernel = client और services में संवाद।

UNIX Commands Facts

① 1. आसान भाषा में

- UNIX एक multi-user OS है। उसमें काम commands से होता है; shell (command पढ़ने वाला program) उन्हें चलाता है।
- Shell script में कुछ special variables हैं जो अक्सर पूछे जाते हैं।

② 2. रोज़मर्रा का example

- Shell = रेस्तराँ का वेटर: तुम order (command) बोलते हो, वह रसोई (kernel) तक पहुँचाता है।

③ 3. step by step

- Special variables: $? = पिछले command का exit status (0 =

सफल); $# = arguments की संख्या; $0 = script का नाम; $ = सारे arguments; $! = पिछले background process का PID।

- seq: `seq FIRST STEP LAST`। `seq 1 2 10` → 1 3 5 7 9।
- wc -l <a >b &: a की lines गिन, b में लिख, background में।
- chmod -R a+x progs: सब files/sub-dirs में सबके लिए execute।
- Signals: INT (Ctrl-C), TERM (साफ रुको), QUIT (core dump), KILL (block नहीं हो सकता)।
- cron daemon: समय से commands चलाता है; हर user की crontab stored रहती है।
- UNIX file system: Super block (i-nodes की संख्या, blocks, free list), i-node (file का हिसाब + block addresses; आमतौर पर 64 bytes), data blocks।
- i-node से max file size: 12 direct + single indirect + double indirect pointers (उदाहरण numericals में)।

④ 4. Exam Point + Trap

- KILL को block/catch नहीं कर सकते।
- i-node 256 bytes नहीं (64 bytes)।
- crontab stored होती है।
- Trap: 'not correct' पूछा हो तो गलत बयान ढूँढो।

⑤ 5. BPSC ऐसे पूछता है (TRE 1.0/2.0/3.0)

- TRE में file को recycle bin में डाले बिना delete करने वाली key पूछी गई थी (Shift+Delete)। ऐसे shortcut सवालों के लिए shortcuts की list याद रखो।

⑥ 6. NET level में ऐसे आता है

- Type: Match the list ($?, $#, $0, $)।
- Type: command का output (seq)।
- Type: 'सही नहीं' (signals, cron, super block)।

7. याद रखो

- $? status, $# गिनती, $0 नाम, $ सब, $! background PID।
- KILL = unblockable; crontab stored।

Numericals — और practice

Practice 1: Deadlock-free n

- सवाल: 10 resources, हर process को अधिकतम 3। अधिकतम n?
- Formula: $R \geq n(m-1)+1 \rightarrow 10 \geq 2n+1 \rightarrow n \leq 4.5 \rightarrow n = 4$।
- Check: n=5 पर 5×2+1 = 11 > 10 → deadlock संभव।

Practice 2: Semaphore

- सवाल: semaphore 4 हैं; 9 P और 6 V हुए। अंतिम मान?
- 4 − 9 + 6 = 11

Practice 3: CPU utilization

- सवाल: 3 process/मिनट आते हैं, हर को 10 सेकंड। Utilization?
- 3 × (10/60) = 0.5 = 50%।

Practice 4: EAT with TLB

- सवाल: TLB (Translation Lookaside Buffer) hit 90%, TLB 10 ns, memory 100 ns। EAT (Effective Access Time)?
- Hit: 110 ns × 0.9 = 99; Miss: 210 ns × 0.1 = 21; कुल 120 ns।

Practice 5: SSTF

- Head 50; requests 40, 60, 45, 90। SSTF: 50→45 (5) →40 (5) →60 (20) →90 (30) = 60।

Practice 6: i-node file size

- Block 1 KB, address 4 bytes, 10 direct + 1 single indirect। एक block में 1024/4 = 256 pointers। Max = 10×1 KB + 256×1 KB =

266 KB1

Abbreviations — Full Forms

① OS Part 3

- OS = Operating System
- CPU = Central Processing Unit
- CS = Critical Section
- 2PL = Two-Phase Locking
- SSTF = Shortest Seek Time First
- SCAN/C-SCAN = Elevator / Circular SCAN
- LRU = Least Recently Used
- EAT = Effective Access Time
- TLB = Translation Lookaside Buffer
- PID = Process IDentifier
- POSIX = Portable Operating System Interface
- UGC NET = University Grants Commission National Eligibility Test
- TRE = Teacher Recruitment Exam
- BPSC = Bihar Public Service Commission
- wc = word count
- chmod = change mode

Questions + हर सवाल के नीचे Explanation

Q1. [UGC NET Jun 2012 (Paper 3)]

Consider the methods used by processes P1 and P2 for accessing their critical sections. The initial values of shared Boolean variables S1 and S2 are randomly assigned,

P1 P2

```
While (S1 = = S2); while (S1 = = S2);
```

Critical section critical section

```
S1 = S2; S1 = S2;
```

Which one of the following statements describes the properties achieved?

(A) Mutual exclusion but not progress
(B) Progress but not mutual exclusion
(C) Neither mutual exclusion nor progress
(D) Both mutual exclusion and progress

हर **option** समझो:

(A) ✔ सही: एक बार में एक ही process अंदर जाता है, पर बारी बारी (alternation) की मजबूरी से Progress टूटता है।
(B) Progress तब होता जब बाहर बैठा process दूसरे को न रोके; यहाँ रोकता है, और Mutual Exclusion तो मिल रही है।
(C) Mutual Exclusion (एक समय में एक) तो मिल रहा है, इसलिए 'दोनों नहीं' गलत।
(D) Progress नहीं मिलता क्योंकि strict alternation है; इसलिए 'दोनों मिले' गलत।

Ans: (A) Mutual exclusion but not progress

इस तरह के सवाल में P1 तब अंदर जाता है जब S1 और S2 बराबर न हों और P2 तब जाता है जब बराबर हों (असली पेपर में P2 की line 'while (S1 != S2)' थी)। दोनों की शर्त उल्टी है, इसलिए एक ही समय पर दोनों अंदर नहीं जा सकते = Mutual Exclusion (एक समय में एक ही process) पक्का। पर अगर P1 को बार-बार अंदर जाना हो तो उसे P2 की बारी का इंतज़ार करना पड़ता है = Progress नहीं। BPSC/NET key: A।

Q2. [UGC NET Jun 2014 (Paper 3)]

Which of the following concurrency protocol ensures both conflict serializability and freedom from deadlock : I. 2-phase locking II. Time phase ordering

(A) Both I & II
(B) II only
(C) I only
(D) Neither I nor II

हर **option** समझो:

(A) 2PL deadlock-free नहीं है, इसलिए I सही नहीं; 'दोनों' गलत।
(B) ✔ सही: Timestamp ordering = conflict serializable + deadlock-free।
(C) 2PL में lock का wait होता है, इसलिए deadlock संभव है; सिर्फ I गलत।
(D) Timestamp ordering दोनों गुण देता है, इसलिए 'कोई नहीं' गलत।

Ans: (B) II only

2PL (Two-Phase Locking) में lock के लिए इंतज़ार होता है, इसलिए deadlock (दो transactions का एक-दूसरे का इंतज़ार) हो सकता है। Timestamp Ordering में कोई इंतज़ार नहीं होता, transaction सीधे abort/restart होती है, इसलिए deadlock नहीं होता और conflict serializable रहता है। सही: सिर्फ II।

Q3. [UGC NET Jun 2014 (Paper 3)]

Consider a uniprocessor system where new processes arrive at an average of five processes per minute and each process needs an average of 6 seconds of service time. What will be the CPU utilization ?

(A) 80 %
(B) 50 %
(C) 60 %
(D) 30 %

हर **option** समझो:

(A) 80% तब होता जब काम 48 सेकंड/मिनट होता; यहाँ 30 सेकंड है।
(B) ✔ सही: 30 सेकंड ÷ 60 सेकंड = 50%।
(C) 60% के लिए 36 सेकंड काम चाहिए, जो नहीं है।
(D) 30% उसे कहते जो 30 सेकंड को सीधे % समझ ले; मिनट में बदलना भूलना trap है।

Ans: (B) 50 %

CPU Utilization (CPU कितने % busy रहा) = arrival rate × service time। यहाँ 5 process/मिनट × 6 सेकंड = 30 सेकंड काम हर 60 सेकंड में = 50%।

Step-by-step Solution:

1. Formula: CPU utilization (ρ) = λ (arrival rate) × S (service time)
2. λ = 5 process प्रति मिनट; S = 6 सेकंड = 6/60 = 0.1 मिनट
3. ρ = 5 × 0.1 = 0.5
4. 0.5 × 100 = 50% → option B
5. Trap: units एक जैसे (सेकंड या मिनट) करना न भूलें।

Q4. [UGC NET Jun 2014 (Paper 3)]

Which of the following statements is not true about disk-arm scheduling algorithms ?

(A) SSTF (shortest seek time first) algorithm increases performance of FCFS.
(B) The number of requests for disk service are not influenced by file allocation method.
(C) Caching the directories and index blocks in main memory can also help in reducing disk arm movements.
(D) SCAN and C-SCAN algorithms are less likely to have a starvation problem.

हर **option** समझो:

(A) यह बात सही है: SSTF में हेड पास वाली request पहले लेता है, इसलिए FCFS से तेज़।
(B) ✔ सही (यही गलत बयान है): allocation method से requests की संख्या बदलती है।
(C) Directory/index को memory में cache करने से disk पर आना कम होता है; यह बयान सही है।
(D) SCAN/C-SCAN में हेड पूरी disk पर घूमता है, starvation की संभावना कम; बयान सही।

Ans: (B) The number of requests for disk service are not influenced by file allocation method.

SSTF (Shortest Seek Time First) FCFS से बेहतर चलता है, पर starvation दे सकता है। SCAN/C-SCAN में starvation कम होता है। पर 'file allocation method disk requests की संख्या पर असर नहीं डालता' गलत है, क्योंकि contiguous/linked/indexed का तरीका ही तय करता है कि कितने disk access लगेंगे। इसलिए 'not true' वाला option B है।

Q5. [UGC NET Jun 2014 (Paper 3)]

A part of Windows 2000 operating system that is not portable is

(A) Device Management
(B) Virtual Memory Management
(C) Processor Management
(D) User Interface

हर **option** समझो:

(A) Device drivers अलग-अलग होते हैं, पर key इसे नहीं मानती।
(B) ✔ सही: key के अनुसार VM Management hardware-dependent, इसलिए non-portable।
(C) Processor (scheduling) management portable माना गया।
(D) User Interface portable हिस्सा है।

Ans: (B) Virtual Memory Management

Windows 2000 का जो हिस्सा hardware पर निर्भर है वह portable नहीं होता। UGC NET key के अनुसार Virtual Memory Management (page table की बनावट processor पर निर्भर)

portable नहीं है। यह एक याद करने वाला fact है।

Q6. [UGC NET Jun 2014 (Paper 3)]

Match the following with reference to Unix shell scripts : List – I List – II a. $? i. File name of the current script b. $# ii. List of arguments c. $0 iii. The number of arguments d. $* iv. Exit status of last command Codes : a b c d

(A) iii ii i iv
(B) ii iii i iv
(C) iv iii i ii
(D) i iii i iv

हर **option** समझो:

(A) $# 'ii' नहीं है; यहाँ a और b का मेल ही उल्टा है।
(B) $? को 'ii' (list) से जोड़ना गलत है।
(C) ✔ सही: a→iv, b→iii, c→i, d→ii।
(D) यह option में 'i' दो बार है; ऐसा नहीं हो सकता।

Ans: (C) iv iii i ii

UNIX shell script के special variables: $? = पिछले command का exit status (0 मतलब सफल), $# = arguments की गिनती, $0 = script का नाम, $ = सारे arguments की list।

Q7. [UGC NET Jun 2014 (Paper 3)]

The output generated by the LINUX command : $ seq 1 2 10 will be

(A) 1 2 10
(B) 1 2 3 4 5 6 7 8 9 10
(C) 1 3 5 7 9
(D) 1 5 10

हर **option** समझो:

(A) Beginning से end तक सीधे नहीं छापता; बीच वाला 2 step है।
(B) 1 से 10 तक सारे नंबर तब आते जब step 1 होता (seq 1 10)।
(C) ✔ सही: 1,3,5,7,9 (अगला 11 > 10 इसलिए रुकता है)।
(D) 1 5 10 जैसा कोई pattern नहीं बनता।

Ans: (C) 1 3 5 7 9

`seq FIRST INCREMENT LAST` — यहाँ 1 से शुरू, 2 के step से, 10 तक: 1 3 5 7 9। बीच का नंबर step होता है, end नहीं।

Q8. [UGC NET Dec 2014]

Consider the following justifications for commonly using the two-level CPU scheduling : I. It is used when memory is too small to hold all the ready processes. II. Because its performance is same as that of the FIFO. III. Because it facilitates putting some set of processes into memory and a choice is made from that. IV. Because it does not allow to adjust the set of in-core processes. Which of the following is true ?

(A) I, III and IV
(B) I and II
(C) III and IV
(D) I and III

हर **option** समझो:

(A) IV गलत है (यह set adjust करने देता है), इसलिए I,III,IV नहीं।
(B) II गलत है, FIFO जैसा performance नहीं।
(C) IV गलत होने से III,IV सही नहीं।
(D) ✔ सही: I और III ही सही हैं।

Ans: (D) I and III

Two-level scheduling तब काम आता है जब memory सारे ready processes के लिए छोटी हो: कुछ processes memory में रखे जाते हैं और उन्हीं में से चुनाव होता है (I और III)। इसका performance FIFO जैसा नहीं होता, और यह in-core set को adjust करने देता है (IV गलत)।

Q9. [UGC NET Dec 2014]

Which of the following conditions does not hold good for a solution to a critical section problem ?

(A) No assumptions may be made about speeds or the number of CPUs.
(B) No two processes may be simultaneously inside their critical sections.
(C) Processes running outside its critical section may block other processes.
(D) Processes do not wait forever to enter its critical section.

हर **option** समझो:

(A) यह शर्त सही है (कोई speed assumption नहीं)।
(B) यह Mutual Exclusion है, सही शर्त।
(C) ✔ सही उत्तर (यही शर्त नहीं है): Progress कहता है बाहर वाला दूसरों को न रोके।
(D) यह Bounded Waiting है, सही शर्त।

Ans: (C) Processes running outside its critical section may block other processes.

Critical Section (ऐसा code हिस्सा जहाँ shared data छुआ जाता है) की शर्तें: Mutual Exclusion (एक समय में एक), Progress (बाहर वाला दूसरे को न रोके), Bounded Waiting (कोई हमेशा इंतज़ार न करे), और speed/CPU संख्या की कोई मान्यता नहीं। 'बाहर वाले process रोक सकते हैं' शर्त नहीं बल्कि उसके उल्टी बात है।

Q10. [UGC NET Dec 2014]

In a demand paging memory system, page table is held in registers. The time taken to service a page fault is 8 m.sec. if an empty frame is available or if the replaced page is not modified, and it takes 20 m.secs., if the replaced page is modified. What is the average access time to service a page fault assuming that the page to be replaced is modified 70% of the time ?

(A) 11.6 m.sec.
(B) 16.4 m.sec.
(C) 28 m.sec.
(D) 14 m.sec.

हर **option** समझो:

(A) 11.6 तब आता जब 30% modified होता (उल्टा लगाया)।
(B) ✔ सही: 0.7×20 + 0.3×8 = 14 + 2.4 = 16.4।
(C) 28 सिर्फ 8+20 जोड़ना है, probability नहीं लगाई।
(D) 14 में सिर्फ 0.7×20 है, 0.3×8 जोड़ना भूल गए।

Ans: (B) 16.4 m.sec.

Page fault का average service time = (modified होने की probability × 20 ms) + (not modified की × 8 ms)। यहाँ 70% modified: 0.7×20 + 0.3×8 = 16.4 ms।

Step-by-step Solution:

1. Formula: Average time = P(modified) × T(modified) + P(not modified) × T(not modified)
2. P(modified)=0.7, T=20 ms → 0.7×20 = 14 ms
3. P(not modified)=0.3, T=8 ms → 0.3×8 = 2.4 ms
4. Total = 14 + 2.4 = 16.4 ms → option B
5. Trap: दोनों probabilities जोड़कर 1 होनी चाहिए।

Q11. [UGC NET Dec 2014 (Paper 3)]

Which methods are utilized to control the access to an object in multi-threaded programming ?

(A) Asynchronized methods
(B) Synchronized methods
(C) Serialized methods
(D) None of the above

हर **option** समझो:

(A) 'Asynchronized' जैसा कोई चलन नहीं।
(B) ✔ सही: synchronized keyword object पर lock लगाता है।
(C) Serialization object को file/stream में बदलने के लिए है, access control के लिए नहीं।
(D) synchronized सही है, इसलिए 'कोई नहीं' गलत।

Ans: (B) Synchronized methods

Multi-threaded programming (Java) में एक object को एक समय में एक ही thread चलाए, इसके लिए synchronized methods/blocks का उपयोग होता है। इसमें lock (monitor) लगता है।

Q12. [UGC NET Dec 2014 (Paper 3)]

Consider an imaginary disk with 40 cylinders. A request come to read a block on cylinder 11. While the seek to cylinder 11 is in progress, new requests come in for cylinders 1, 36, 16, 34, 9 and 12 in that order. The number of arm motions using shortest seek first algorithm is

(A) 111
(B) 112
(C) 60
(D) 61

हर **option** समझो:

(A) 111 FCFS जैसा बड़ा आंकड़ा है, SSTF में कम आता है।
(B) 112 भी बहुत बड़ा है।
(C) 60 में एक step 1 कम गिन लिया।
(D) ✔ सही: 1+3+7+15+33+2 = 61।

Ans: (D) 61

SSTF (Shortest Seek Time First): हेड हमेशा अभी की position से सबसे पास वाली request लेता है। शुरुआत 11 से। कुल arm motion = सारे steps का योग = 61।

Step-by-step Solution:

1. Start = cylinder 11; requests: 1, 36, 16, 34, 9, 12
2. 11 से सबसे पास 12 (दूरी 1) □ 12
3. 12 से सबसे पास 9 (3) □ 9
4. 9 से सबसे पास 16 (7) □ 16

5. 16 से: 1 की दूरी 15, 34 की 18 □ 1 (15)
6. 1 से 34 (33)
7. 34 से 36 (2)
8. Total = 1+3+7+15+33+2 = 61 → option D
9. Trap: हर step पर 'अभी की position' से सबसे पास चुनें, पूरी list से नहीं।

Q13. [UGC NET Dec 2014 (Paper 3)]

Monitor is an Interprocess Communication (IPC) technique which can be described as

(A) It is higher level synchronization primitive and is a collection of procedures, variables, and data structures grouped together in a special package.

(B) It is a non-negative integer which apart from initialization can be acted upon by wait and signal operations.

(C) It uses two primitives, send and receive which are system calls rather than language constructs.

(D) It consists of the IPC primitives implemented as system calls to block the process when they are not allowed to enter critical region to save CPU time.

हर **option** समझो:

(A) ✔ सही: यह Monitor की परिभाषा है।

(B) यह Semaphore की परिभाषा है (integer + wait/signal)।

(C) यह Message Passing है (send/receive)।

(D) यह sleep/wakeup जैसी primitives हैं, Monitor नहीं।

Ans: (A) It is higher level synchronization primitive and is a collection of procedures, variables, and data structures grouped together in a special package.

Monitor एक high-level synchronization tool है: procedures, variables और data structures एक package में, और एक समय में एक ही process monitor के अंदर रह सकता है। Semaphore = non-negative integer (wait/signal), Message passing = send/receive।

Q14. [UGC NET Dec 2014 (Paper 3)]

Which of the following versions of Windows O.S. contain built-in partition manager which allows us to shrink and expand pre-defined drives ?

(A) Windows Vista
(B) Windows 2000
(C) Windows NT
(D) Windows 98

हर **option** समझो:

(A) ✔ सही: Vista में shrink/expand volume मिला।
(B) Windows 2000 में partition बड़ा/छोटा करने की सुविधा नहीं थी।
(C) NT में भी नहीं थी।
(D) 98 में नहीं थी (FDISK से हटाकर नया बनाना पड़ता)।

Ans: (A) Windows Vista

Windows Vista में पहली बार built-in disk partition manager (Disk Management में Shrink/Extend Volume) आया। 98, 2000, NT में यह सुविधा नहीं थी।

Q15. [UGC NET Jul 2016 (Paper 3)]

In Unix operating system, when a process creates a new process using the fork () system call, which of the following state is shared between the parent process and child process ?

(A) Heap
(B) Stack
(C) Shared memory segments
(D) Both Heap and Stack

हर **option** समझो:

(A) Heap की copy बनती है, shared नहीं।
(B) Stack की भी copy बनती है।
(C) ✔ सही: shared memory segment दोनों के लिए एक ही रहता है।
(D) Heap और Stack दोनों अलग होते हैं।

Ans: (C) Shared memory segments

fork() से child बनने पर child को parent के stack और heap की अलग कॉपी मिलती है। पर shared memory segments parent-child दोनों के बीच साझा रहते हैं।

Q16. [UGC NET Jul 2016 (Paper 3)]

Which of the following information about the UNIX file system is not correct ?

(A) Super block contains the number of i-nodes, the number of disk blocks, and the start of the list of free disk blocks.
(B) An i-node contains accounting information as well as enough information to locate all the disk blocks that holds the file's data.
(C) Each i-node is 256-bytes long.
(D) All the files and directories are stored in data blocks.

हर **option** समझो:

(A) यह बयान सही है (Super block का काम)।
(B) i-node में accounting और block addresses दोनों होते हैं; सही बयान।
(C) ✔ सही (गलत बयान): i-node 256 bytes का नहीं होता।
(D) फाइलें और directories data blocks में ही रहती हैं, सही।

Ans: (C) Each i-node is 256-bytes long.

UNIX file system: Super block में i-nodes की संख्या, blocks की संख्या और free blocks की list शुरू होती है। i-node में file का हिसाब और data blocks का पता होता है। पर i-node की size 256 bytes नहीं, परंपरागत रूप से 64 bytes होती है। Files और directories data blocks में रहते हैं।

Q17. [UGC NET Jul 2016 (Paper 3)]

Which of the following option with reference to UNIX operating system is not correct ?

(A) INT signal is sent by the terminal driver when one types <Control-C> and it is a request to terminate the current operation.
(B) TERM is a request to terminate execution completely. The receiving process will clean up its state and exit.
(C) QUIT is similar to TERM, except that it defaults to producing a core dump if not caught.
(D) KILL is a blockable signal.

हर **option** समझो:

(A) INT Ctrl-C से जाता है, सही बयान।
(B) TERM clean exit माँगता है, सही बयान।
(C) QUIT core dump बनाता है, सही बयान।
(D) ✔ सही (गलत बयान): KILL blockable नहीं है।

Ans: (D) KILL is a blockable signal.

UNIX signals: INT (Ctrl-C) रोकने की request; TERM साफ-सफाई करके रुकने की request; QUIT = TERM + core dump; KILL को block या catch नहीं किया जा सकता (यह पक्का मारता है)। इसलिए 'KILL blockable है' गलत है।

Q18. [UGC NET Jul 2016 (Paper 3)]

What is the function of following UNIX command ? WC – l <a > b &

(A) It runs the word count program to count the number of lines in its input, a, writing the result to b, as a foreground process.
(B) It runs the word count program to count the number of lines in its input, a, writing the result to b, but does it in the background.
(C) It counts the errors during the execution of a process, a, and puts the result in process b.
(D) It copies the 'l' numbers of lines of program from file, a, and stores in file b.

हर **option** समझो:
(A) & का मतलब background है, foreground नहीं।
(B) ✔ सही: lines गिनता है, input a, output b, background।
(C) यह errors नहीं गिनता।
(D) यह lines copy नहीं करता, गिनता है।

Ans: (B) It runs the word count program to count the number of lines in its input, a, writing the result to b, but does it in the background.

`wc -l <a >b &` : wc = word count, -l = lines गिनो, <a = a से input लो, >b = result b में लिखो, & = background में चलाओ।

Q19. [UGC NET Jul 2016 (Paper 3)]

Which of the following statement is not correct with reference to cron daemon in UNIX O.S. ?

(A) The cron daemon is the standard tool for running commands on a pre-determined schedule.
(B) It starts when the system boots and runs as long as the system is up.
(C) Cron reads configuration files that contain list of command lines and the times at which they invoked.
(D) Crontab for individual users are not stored.

हर **option** समझो:
(A) यह cron की परिभाषा है, सही।
(B) यह भी सही बयान है।
(C) ✔ सही बयान: config files में commands और समय होते हैं।
(D) ✔ सही (गलत बयान): users की crontab files stored रहती हैं।

Ans: (D) Crontab for individual users are not stored.

cron daemon (हमेशा चलने वाला service) निर्धारित समय पर commands चलाता है; boot पर शुरू

होता है; config files पढ़ता है। हर user की crontab file स्टोर की जाती है, इसलिए 'crontab stored नहीं होती' गलत है।

Q20. [UGC NET Dec 2018]

Suppose a system has 12 instances of some resources with n processes competing for that resource. Each process may require 4 instances of the resource. The maximum value of n for which the system never enters into deadlock is

(A) 3
(B) 4
(C) 5
(D) 6

हर **option** समझो:

(A) ✔ सही: n=3 पर 3×3+1 = 10 □ 12 ठीक है।
(B) n=4 पर 4×3+1 = 13 > 12, deadlock संभव।
(C) n=5 पर 16 > 12, deadlock संभव।
(D) n=6 पर 19 > 12, deadlock संभव।

Ans: (A) 3

Deadlock से बचने का नियम: n processes, हर को अधिकतम m resources चाहिए, कुल R resources। deadlock नहीं होगा अगर R ≥ n×(m−1) + 1। यहाँ 12 □ n×3 + 1 → n ≤ 3.67 □ अधिकतम n = 3।

Step-by-step Solution:

1. Rule: deadlock-free अगर R ≥ n×(m−1) + 1
2. R (total resources) = 12, m (हर process की max जरूरत) = 4
3. Worst case: हर process को 3 मिल गए और सब 1 और का इंतज़ार कर रहे हैं
4. n×3 + 1 ≤ 12 → n×3 ≤ 11 → n ≤ 3 (पूर्णांक)
5. Answer: n = 3 → option A
6. Trap: n=4 पर 12 resources ठीक बँट जाते हैं (3-3-3-3), पर सब 1 और माँगें तो deadlock हो जाता है।

Q21. [UGC NET Jun 2019]

A computer has six tapes drives with n processes competing for them. Each process may need two drives. What is the maximum value of n for the system to be deadlock free?

(A) 5
(B) 4
(C) 3
(D) 6

हर **option** समझो:

(A) ✔ सही: n=5 पर 5×1+1 = 6 □ 6।

(B) n=4 भी deadlock-free है, पर 'maximum' पूछा है, इसलिए 5।
(C) n=3 सुरक्षित है, पर maximum नहीं।
(D) n=6 पर हर process को 1 drive मिले और सब दूसरी का इंतज़ार करें, deadlock।

Ans: (A) 5

वही नियम: R ≥ n×(m−1)+1। यहाँ R=6 tape drives, m=2। 6 □ n×1 + 1 → n ≤ 5। अधिकतम n = 5।

Step-by-step Solution:

1. Rule: R ≥ n×(m−1)+1
2. R=6, m=2
3. 6 ≥ n×(2−1)+1 = n+1
4. n ≤ 5
5. Answer: 5 → option A
6. Trap: 'deadlock नहीं' और 'maximum' दोनों शब्द देखें।

Q22. [UGC NET Dec 2018]

Suppose P,Q and R are co-operating processes satisfying Mutual Exclusion condition. Then if the process Q is executing in its critical section then

(A) 'P' executes in critical section
(B) 'R' executes in critical section
(C) Neither 'P' nor 'Q' executes in their critical section
(D) Both 'P' and 'R' executes in critical section

हर **option** समझो:

(A) Q के अंदर रहते P अंदर नहीं जा सकता।
(B) R भी अंदर नहीं जा सकता।
(C) ✔ सही: न P न R, यानी Q अकेला है।
(D) P और R दोनों नहीं जा सकते।

Ans: (C) Neither 'P' nor 'Q' executes in their critical section

Mutual Exclusion का मतलब: जब Q अपने critical section में है, तब P और R में से कोई भी अपने critical section में नहीं हो सकता।

Q23. [UGC NET Jun 2023]

At particular time of computation, the value of a counting semaphore is 7.Then 20p operations and 'x' V operations were completed on this semaphore.if the final value of semaphore is 5,x will be

(A) 15
(B) 22
(C) 18
(D) 13

हर **option** समझो:

(A) 15 लेने पर मान 2 आता है, 5 नहीं।
(B) 22 लेने पर मान 9 आता है।
(C) ✔ सही: 7−20+18 = 5।
(D) 13 लेने पर मान 0 आता है।

Ans: (C) 18

Counting semaphore (गिनती वाला signal): P (wait) operation मान 1 घटाता है, V (signal) 1 बढ़ाता है। शुरू 7, 20 P → 7−20, x V → +x, अंत 5: 7 − 20 + x = 5 → x = 18।

Step-by-step Solution:

1. P (wait) operation: मान −1; V (signal): मान +1
2. शुरुआती मान = 7
3. 20 P के बाद: 7 − 20 = −13
4. x V के बाद: −13 + x = 5
5. x = 5 + 13 = 18 → option C
6. Trap: मान negative जा सकता है (negative = इंतज़ार करते processes की गिनती)।

Q24. [UGC NET Dec 2021-Jun 2022]

If an operating system does not allow a child process to exist when the parent process has been terminated, this phenomenon is calls as-

(A) Threading
(B) Cascading termination
(C) Zombie termination
(D) Process killing

हर **option** समझो:

(A) Threading अलग concept है (एक process के अंदर कई धागे)।
(B) ✔ सही: parent खत्म तो सारे child भी खत्म।
(C) Zombie वह है जो खत्म हो गया पर parent ने उसकी status नहीं पढ़ी।
(D) 'Process killing' कोई standard term नहीं।

Ans: (B) Cascading termination

अगर parent के खत्म होने पर OS child को भी खत्म कर देता है तो इसे Cascading Termination कहते हैं। UNIX में child अनाथ (orphan) रहकर चलता है, पर कुछ OS ऐसा नहीं होने देते।

Q25. [UGC NET Dec 2022]

In Linux operating system, when _____ is invoked, it is passed a set of flags that determine how much sharing is to take place between the parent and child tasks.

(A) fork()
(B) clone()
(C) pthread()
(D) thread()

हर **option** समझो:

(A) fork() में कोई flag नहीं, पूरी कॉपी बनती है।
(B) ✔ सही: clone() को flags देकर sharing तय होती है।
(C) pthread library है, system call नहीं (और यह flags नहीं लेता)।
(D) thread() नाम का कोई system call नहीं।

Ans: (B) clone()

Linux में clone() system call से thread या process बनता है। इसके साथ flags दिए जाते हैं जो तय करते हैं कि parent-child के बीच कितना कुछ share होगा (memory, files, signals)। fork() में flags नहीं होते।

Q26. [UGC NET Dec 2022]

The main function of the microkernel is to provide a communication facility between the _____ program and the various _____ that are also running in user space.

(A) Virtual, Processes
(B) System, Processes
(C) Client, Services
(D) Virtual, Services

हर **option** समझो:

(A) 'Virtual, Processes' जैसा कोई pair नहीं।
(B) 'System, Processes' भी सही pair नहीं।
(C) ✔ सही: client program और services के बीच।
(D) 'Virtual, Services' गलत।

Ans: (C) Client, Services

Microkernel (छोटा kernel) का मुख्य काम client program और उन services के बीच communication करना है जो user space में चलती हैं (जैसे file server)। Message passing इसके लिए है।

Q27. [UGC NET Jun 2023]

Which of the following concurrency control protocols ensure both conflict serializability and freedom from deadlock?
Statement I: Two phase locking
Statement II: Timestamp ordering
In the light of the above statements, choose the most appropriate answer from the options given below:

(A) Both statement I and statement II are correct
(B) Both statement I and statement II are incorrect
(C) Statement I is correct but Statement Ii is incorrect
(D) Statement I is incorrect but Statement Ii is correct

हर **option** समझो:
(A) I गलत होने से 'दोनों सही' नहीं।
(B) II सही है, इसलिए 'दोनों गलत' नहीं।
(C) I (2PL) deadlock-free नहीं, इसलिए यह गलत।
(D) ✔ सही: I गलत और II सही।

Ans: (D) Statement I is incorrect but Statement Ii is correct

Statement I (2PL) गलत है क्योंकि 2PL में deadlock हो सकता है। Statement II (Timestamp ordering) सही है, क्योंकि इसमें कोई लॉक-इंतज़ार नहीं होता और यह conflict serializable व deadlock-free है। इसलिए I गलत, II सही।

Q28. [UGC NET Dec 2021-Jun 2022]

Match List I with List II:
List I List II
(A) Stack Algorithm (I) Deadloack
(B) Elevator algorithm (II) Disk Scheduling
(C) Priority scheduling algorithm (III) Page replacement
(D) Havender algorithm (IV) CPU Scheduling
CHoose the correct answer from the options given below:

(A) (A)-(III), (B)-(II), (C)-(IV), (D)-(I)
(B) (A)-(II), (B)-(III), (C)-(IV), (D)-(I)
(C) (A)-(III), (B)-(II), (C)-(I), (D)-(IV)
(D) (A)-(II), (B)-(III), (C)-(I), (D)-(IV)

हर **option** समझो:
(A) ✔ सही: A-III, B-II, C-IV, D-I।
(B) Stack algorithm disk scheduling नहीं है।
(C) Priority को Deadlock से जोड़ना गलत।
(D) Stack और Elevator दोनों उल्टे जुड़े हैं।

Ans: (A) (A)-(III), (B)-(II), (C)-(IV), (D)-(I)

Match: Stack Algorithm = Page replacement (LRU, Optimal जैसे); Elevator algorithm = Disk scheduling (SCAN); Priority = CPU Scheduling; Havender algorithm = Deadlock (prevention की पद्धति)।

Q29. [UGC NET Nov 2020]

Consider a single-level page table system, with the page table stored in the memory. If the hit rate to TLB is 80%, and it takes 15 nanoseconds to search the TLB, and 150 nanoseconds to access the main memory, then what is the effective memory access time, in nanoseconds?

(A) 185
(B) 195
(C) 205
(D) 175

हर **option** समझो:

(A) 185 में miss वाला extra memory access ठीक नहीं गिना।
(B) ✔ सही: 132 + 63 = 195 ns।
(C) 205 में TLB time miss पर दो बार गिन लिया।
(D) 175 में TLB search time हटा दिया।

Ans: (B) 195

Single-level page table memory में है: TLB hit पर सिर्फ 1 memory access; miss पर page table के लिए 1 और data के लिए 1। EAT (Effective Access Time) = h×(T+M) + (1−h)×(T+M+M)।

Step-by-step Solution:

1. Formula: EAT = h × (T + M) + (1 − h) × (T + 2M)
2. h (hit ratio) = 0.8, T (TLB search time) = 15 ns, M (memory access) = 150 ns
3. Hit: 15 + 150 = 165 ns → 0.8 × 165 = 132
4. Miss: 15 + 150 (page table) + 150 (data) = 315 ns → 0.2 × 315 = 63
5. EAT = 132 + 63 = 195 ns → option B
6. Trap: miss में TLB search भी लगता है, और memory दो बार।

Q30. [UGC NET Jun 2025]

What is the total swap time (Swap in & Swap out) in a system for a 15 MB process with a transfer rate of 30 MBps, given an average latency of 12 ms (no head seeks involved)?

(A) 1.024 sec
(B) 1.00 sec
(C) 0.512 sec
(D) 12 sec

हर **option** समझो:

(A) ✔ सही: 1.0 s + 0.024 s = 1.024 s।
(B) 1.00 s में latency जोड़ना भूल गए।
(C) 0.512 s केवल एक तरफ का swap है।
(D) 12 s बिलकुल गलत (units की गड़बड़)।

Ans: (A) 1.024 sec

Swap time = (process size ÷ transfer rate) एक बार बाहर, एक बार अंदर + हर बार latency। 15 MB ÷ 30 MBps = 0.5 s। Swap out + swap in = 1.0 s। Latency 12 ms × 2 = 24 ms। Total = 1.024 s।

Step-by-step Solution:

1. Transfer time (एक तरफ) = size ÷ rate = 15 MB ÷ 30 MBps = 0.5 s
2. Swap out + swap in = 2 × 0.5 = 1.0 s
3. Latency एक बार = 12 ms; दो बार = 24 ms = 0.024 s
4. Total = 1.0 + 0.024 = 1.024 s → option A
5. Trap: swap दो तरफ का होता है (out और in)।

Q31. [UGC NET Jul 2018]

Consider a virtual page reference string 1, 2, 3, 2, 4, 2, 5, 2, 3, 4. Suppose LRU page replacement algorithm is implemented with 3 page frames in main memory. Then the number of page faults are

(A) 5
(B) 7
(C) 9
(D) 10

हर **option** समझो:

(A) 5 faults बहुत कम हैं।
(B) ✔ सही: कुल 7 faults।
(C) 9 faults ज़्यादा गिन लिए।
(D) 10 में हर reference को fault गिन लिया।

Ans: (B) 7

LRU (Least Recently Used) = सबसे पुराना use हुआ page हटाओ। 3 frames के साथ reference string 1,2,3,2,4,2,5,2,3,4 में 7 page faults आते हैं।

Step-by-step Solution:

1. Frames = 3; fault को F, hit को H लिखें
2. 1 → F [1]; 2 → F [1,2]; 3 → F [1,2,3]
3. 2 → H
4. 4 → F: सबसे पुराना use 1 है, उसे हटाओ □ [4,2,3]
5. 2 → H
6. 5 → F: सबसे पुराना 3 □ [4,2,5]
7. 2 → H
8. 3 → F: सबसे पुराना 4 □ [3,2,5]
9. 4 → F: सबसे पुराना 5 □ [3,2,4]
10. Faults = 1,2,3,4,5('5'),3,4 = 7 → option B

Q32. [UGC NET Jul 2018]

Page information in memory is also called as Page Table. The essential contents in each entry of a page table is/are

(A) Page Access information
(B) Virtual Page number
(C) Page Frame number
(D) Both virtual page number and Page Frame Number

हर **option** समझो:

(A) Access info (R/W bits) ज़रूरी तो हैं पर 'essential' में मुख्य frame number है।
(B) Virtual page number index से मिलता है, entry में नहीं।
(C) ✔ सही: frame number (physical address बनाने के लिए)।
(D) VPN store नहीं होता, इसलिए 'दोनों' गलत।

Ans: (C) Page Frame number

Page table की हर entry में ज़रूरी चीज़ Page Frame number है (यानी वह page RAM के किस frame में है)। Virtual page number तो entry का index (क्रम) होता है, अलग से store नहीं होता।

Q33. [UGC NET Jul 2018]

Which UNIX/Linux command is used to make all files and sub-directories in the directory "progs" executable by all users ?

(A) chmod− R a+x progs
(B) chmod −R 222 progs
(C) chmod−X a+x progs
(D) chmod −X 222 progs

हर **option** समझो:

(A) ✔ सही: -R recursive और a+x सब के लिए execute।

(B) 222 का मतलब write permission है, execute नहीं।
(C) '-X' बड़ा X अलग option है; वैसे भी ठीक syntax नहीं।
(D) 222 write देता है और -X गलत।

Ans: (A) chmod− R a+x progs

`chmod` = permission बदलो, `-R` = recursive (सब sub-folders भी), `a+x` = सबके लिए execute जोड़ो। इसलिए `chmod -R a+x progs`।

Q34. [UGC NET Jun 2025]

Which of the following statements is correct for Pthreads?

(A) It refers to POSIX standard (IEEE 1002.1c)
(B) This standard defines API for thread creation only
(C) This is a specification only for thread behavior and not its implementation
(D) Operating systems like Solaris, Linux, and Mac OS X (except Tru64 UNIX) implement Pthreads

हर **option** समझो:

(A) असली standard IEEE 1003.1c है, option में 1002.1c लिखा है, इसलिए गलत।
(B) API में creation के साथ synchronization भी है, इसलिए 'सिर्फ creation' गलत।
(C) ✔ सही: यह specification है, implementation नहीं।
(D) किताब में Tru64 UNIX भी इसे लागू करने वालों में है; 'except' लिखना गलत है।

Ans: (C) This is a specification only for thread behavior and not its implementation

Pthreads POSIX thread का standard है; यह thread के behavior की specification है, implementation नहीं (Solaris, Linux, Mac OS X इसे लागू करते हैं)। यह सिर्फ creation का API नहीं, synchronization भी देता है। key: C।

Q35. [UGC NET Nov 2020]

Suppose you have a Linux file system where the block size is 2K bytes, a disk address is 32 bits, and i-node contains the disk addresses of the first 12 direct blocks of file, a single indirect block, and a double indirect block. Approximately, what is the largest file that can be represented by an i-node?

(A) 513 Kbytes
(B) 513 MBytes
(C) 537 Mbytes
(D) 537 KBytes

हर **option** समझो:

(A) 513 KB बहुत कम है, double indirect 512 MB देता है।

(B) ✔ सही: 24 KB + 1 MB + 512 MB ≈ 513 MB।
(C) 537 MB किसी और block size से आता है।
(D) 537 KB भी गलत unit/मान है।

Ans: (B) 513 MBytes

i-node में 12 direct blocks, 1 single indirect और 1 double indirect pointer हैं। block = 2 KB, address 32 bit = 4 bytes → एक block में 2048/4 = 512 pointers। Direct = 12×2 KB = 24 KB; single = 512×2 KB = 1 MB; double = 512×512×2 KB = 512 MB। कुल ≈ 513 MB।

Step-by-step Solution:

1. Block = 2 KB = 2048 bytes; address = 32 bit = 4 bytes
2. एक indirect block में pointers = 2048 ÷ 4 = 512
3. Direct: 12 × 2 KB = 24 KB
4. Single indirect: 512 × 2 KB = 1024 KB = 1 MB
5. Double indirect: 512 × 512 × 2 KB = 524288 KB = 512 MB
6. कुल = 24 KB + 1 MB + 512 MB ≈ 513 MB → option B
7. Trap: pointers की संख्या = block size ÷ address size।

Answer Key

Q1 - A	**Q2 - B**	**Q3 - B**	**Q4 - B**	**Q5 - B**
Q6 - C	**Q7 - C**	**Q8 - D**	**Q9 - C**	**Q10 - B**
Q11 - B	**Q12 - D**	**Q13 - A**	**Q14 - A**	**Q15 - C**
Q16 - C	**Q17 - D**	**Q18 - B**	**Q19 - D**	**Q20 - A**
Q21 - A	**Q22 - C**	**Q23 - C**	**Q24 - B**	**Q25 - B**
Q26 - C	**Q27 - D**	**Q28 - A**	**Q29 - B**	**Q30 - A**
Q31 - B	**Q32 - C**	**Q33 - A**	**Q34 - C**	**Q35 - B**